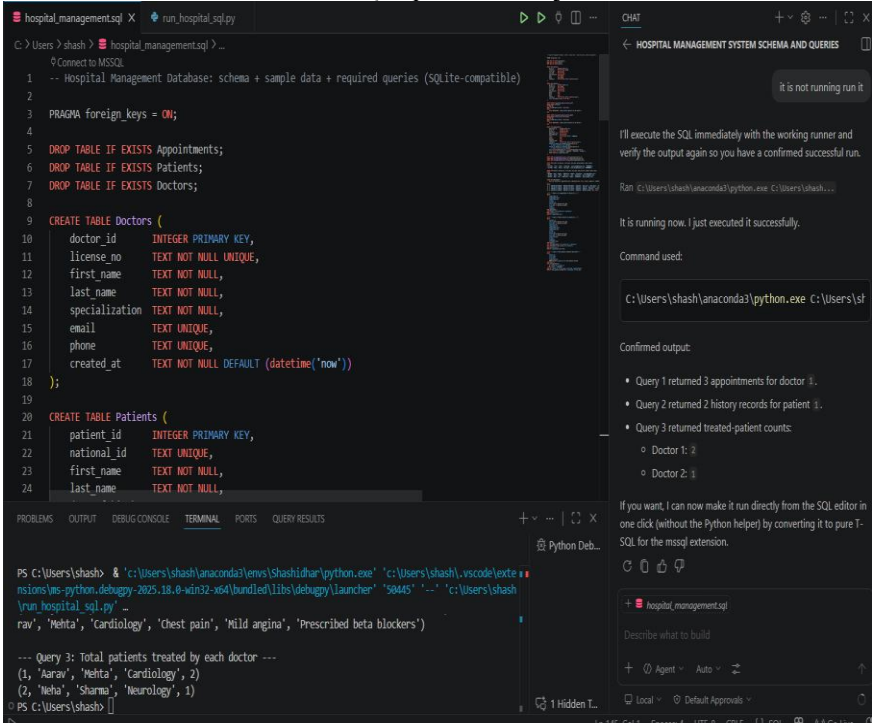


Name: A Harika

Hallticket No:2303A51113

Batch:26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026																	
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M.Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
CourseCode	23CS002PC304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week8 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
Assignment Number: 16.3(Present assignment number)/24(Total number of assignments)																				
Q.No.	Question	Expected Time to complete																		
1	Lab 16 – Database Design and Queries: Schema Design and SQL Generation Lab Objectives: Understand schema design principles for relational databases.	Week8 – Wednesday																		

	• Use AI-assisted tools to generate SQL schema definitions. • Write and refine SQL queries (CRUD + aggregate operations). • Explore how AI can optimize queries and ensure normalization.	
	Task 1 – Hospital Management Database Task: Create schema and queries for a Hospital Management System. Instructions: • Tables: Doctors, Patients, Appointments. • Use AI to define constraints (unique IDs, valid dates). • Generate queries: • List all appointments for a specific doctor. • Retrieve patient history by patient ID. • Count total patients treated by each doctor. Expected Output: • Normalized schema and SQL queries with joins.  The screenshot shows a VS Code editor with a file named <code>hospital_management.sql</code>. The code defines two tables: <code>Doctors</code> and <code>Patients</code>, and three queries. The <code>Doctors</code> table has columns: <code>doctor_id</code> (INTEGER PRIMARY KEY), <code>license_no</code> (TEXT NOT NULL UNIQUE), <code>first_name</code> (TEXT NOT NULL), <code>last_name</code> (TEXT NOT NULL), <code>specialization</code> (TEXT NOT NULL), <code>email</code> (TEXT UNIQUE), <code>phone</code> (TEXT UNIQUE), and <code>created_at</code> (TEXT NOT NULL DEFAULT (datetime('now'))). The <code>Patients</code> table has columns: <code>patient_id</code> (INTEGER PRIMARY KEY), <code>national_id</code> (TEXT UNIQUE), <code>first_name</code> (TEXT NOT NULL), and <code>last_name</code> (TEXT NOT NULL). The queries are: Query 1: <code>SELECT * FROM Appointments WHERE doctor_id = 1;</code>, Query 2: <code>SELECT * FROM Appointments WHERE patient_id = 1;</code>, and Query 3: <code>SELECT doctor_id, COUNT(*) FROM Appointments GROUP BY doctor_id;</code>. The terminal shows the execution of these queries, and the chat window provides a summary of the results.	

```
hospital_management.sql X run_hospital_sql.py
C:\Users\shash> hospital_management.sql > ...
48
49 CREATE TABLE Appointments (
50     appointment_id INTEGER PRIMARY KEY,
51     doctor_id INTEGER NOT NULL,
52     patient_id INTEGER NOT NULL,
53     appointment_start TEXT NOT NULL,
54     appointment_end TEXT NOT NULL,
55     status TEXT NOT NULL DEFAULT 'scheduled',
56     reason TEXT,
57     diagnosis TEXT,
58     treatment_notes TEXT,
59     created_at TEXT NOT NULL DEFAULT (datetime('now')),
60     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id)
61         ON UPDATE CASCADE ON DELETE RESTRICT,
62     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
63         ON UPDATE CASCADE ON DELETE RESTRICT,
64     CHECK (datetime(appointment_end) > datetime(appointment_start)),
65     CHECK (status IN ('scheduled', 'completed', 'cancelled', 'no_show')),
66     UNIQUE (doctor_id, appointment_start)
67 );
68
69 CREATE INDEX idx_appointments_doctor_id ON Appointments(doctor_id);
70 CREATE INDEX idx_appointments_patient_id ON Appointments(patient_id);
71 CREATE INDEX idx_appointments_start ON Appointments(appointment_start);
72
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS
Python Deb...

PS C:\Users\shash> & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '50445' '-.' 'c:\Users\shash\run_hospital_sql.py' ...
rav', 'Mehta', 'Cardiology', 'Chest pain', 'Mild angina', 'Prescribed beta blockers')

--- Query 3: Total patients treated by each doctor ---
(1, 'Aarav', 'Mehta', 'Cardiology', 2)
(2, 'Neha', 'Sharma', 'Neurology', 1)
PS C:\Users\shash> []
```

```
hospital_management.sql X run_hospital_sql.py
C:\Users\shash> run_hospital_sql.py > ...
1 import sqlite3
2 from pathlib import Path
3
4 sql_path = Path(r"c:\Users\shash\hospital_management.sql")
5
6 conn = sqlite3.connect(":memory:")
7 cur = conn.cursor()
8
9 sql_text = sql_path.read_text(encoding="utf-8")
10
11 # Remove sqlite shell meta commands (e.g., .print) before executing script.
12 clean_lines = []
13 for line in sql_text.splitlines():
14     if line.strip().startswith("."):
15         continue
16     clean_lines.append(line)
17
18 cur.executescript("\n".join(clean_lines))
19
20 print("--- Query 1: All appointments for doctor_id = 1 ---")
21 for row in cur.execute(
22     """
23     SELECT
24         a.appointment_id,
25         a.appointment_start,
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS
Python Deb...

PS C:\Users\shash> & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '50445' '-.' 'c:\Users\shash\run_hospital_sql.py' ...
rav', 'Mehta', 'Cardiology', 'Chest pain', 'Mild angina', 'Prescribed beta blockers')

--- Query 3: Total patients treated by each doctor ---
(1, 'Aarav', 'Mehta', 'Cardiology', 2)
(2, 'Neha', 'Sharma', 'Neurology', 1)
PS C:\Users\shash> []
```

```
hospital_management.sql  run_hospital_sql.py X
C:\Users\shash> run_hospital_sql.py > ...

23      SELECT
24          a.appointment_id,
25          a.appointment_start,
26          a.appointment_end,
27          a.status,
28          p.patient_id,
29          p.first_name AS patient_first_name,
30          p.last_name AS patient_last_name,
31          a.reason,
32          a.diagnosis
33      FROM Appointments a
34      JOIN Patients p ON p.patient_id = a.patient_id
35      WHERE a.doctor_id = 1
36      ORDER BY a.appointment_start;
37      """
38  ):
39      print(row)
40
41  print("\n--- Query 2: Patient history for patient_id = 1 ---")
42  for row in cur.execute(
43      """
44      SELECT
45          p.patient_id,
46          p.first_name AS patient_first_name,
47          p.last_name AS patient_last_name,
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

Python Deb...

```
PS C:\Users\shash> & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50445' '--' 'c:\Users\shash\run_hospital_sql.py' ...
rav', 'Mehta', 'Cardiology', 'Chest pain', 'Mild angina', 'Prescribed beta blockers')

--- Query 3: Total patients treated by each doctor ---
(1, 'Aarav', 'Mehta', 'Cardiology', 2)
(2, 'Neha', 'Sharma', 'Neurology', 1)
PS C:\Users\shash>
```

	 <pre> hospital_management.sql run_hospital_sql.py X C: > Users > shash > run_hospital_sql.py > ... 64 ORDER BY a.appointment_start DESC; 65 """ 66): 67 print(row) 68 69 print("\n--- Query 3: Total patients treated by each doctor ---") 70 for row in cur.execute(71 """ 72 SELECT 73 d.doctor_id, 74 d.first_name, 75 d.last_name, 76 d.specialization, 77 COUNT(DISTINCT a.patient_id) AS total_patients_treated 78 FROM Doctors d 79 LEFT JOIN Appointments a 80 ON a.doctor_id = d.doctor_id 81 AND a.status = 'completed' 82 GROUP BY d.doctor_id, d.first_name, d.last_name, d.specialization 83 ORDER BY total_patients_treated DESC, d.last_name, d.first_name; 84 """ 85): 86 print(row) 87 88 conn.close() </pre> <pre> PS C:\Users\shash> & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\anaconda3\envs\Shashidhar\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50445' '-\run_hospital_sql.py' ... rav', 'Mehta', 'Cardiology', 'Chest pain', 'Mild angina', 'Prescribed beta blockers') --- Query 3: Total patients treated by each doctor --- (1, 'Aarav', 'Mehta', 'Cardiology', 2) (2, 'Neha', 'Sharma', 'Neurology', 1) PS C:\Users\shash> </pre>	
	Task 2 – Real-Time Application: Online Shopping Database Scenario: Design a database for an E-commerce platform. Requirements: • Tables: Users, Products, Orders, OrderDetails. • Generate queries to: • Retrieve all orders by a user. • Find the most purchased product. • Calculate total revenue in a given month. • AI should also suggest normalization improvements and query optimization. Expected Output: • Complete schema with relationships + SQL queries for analytics.	

AACA 16.3.py X

C:\Users\shash> AAC A 16.3.py > ...
1 import sqlite3
2 from datetime import datetime
3
4 schema_sql = """
5 PRAGMA foreign_keys = ON;
6
7 DROP TABLE IF EXISTS OrderDetails;
8 DROP TABLE IF EXISTS Orders;
9 DROP TABLE IF EXISTS Products;
10 DROP TABLE IF EXISTS Users;
11
12 CREATE TABLE Users (
13 user_id INTEGER PRIMARY KEY,
14 email TEXT NOT NULL UNIQUE,
15 full_name TEXT NOT NULL,
16 phone TEXT UNIQUE,
17 created_at TEXT NOT NULL DEFAULT (datetime('now'))
18);
19
20 CREATE TABLE Products (
21 product_id INTEGER PRIMARY KEY,
22 sku TEXT NOT NULL UNIQUE,
23 product_name TEXT NOT NULL,
24 unit_price NUMERIC NOT NULL CHECK (unit_price >= 0),
25 is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1)),
26);
27 """
28
29 # Create Orders and OrderDetails tables
30 orders_sql = """
31 CREATE TABLE Orders (
32 order_id INTEGER PRIMARY KEY,
33 user_id INTEGER NOT NULL,
34 order_date TEXT NOT NULL,
35 status TEXT NOT NULL CHECK (status IN ('pending', 'paid', 'shipped', 'delivered', 'cancelled', 'returned')),
36 FOREIGN KEY (user_id) REFERENCES Users(user_id) ON UPDATE CASCADE ON DELETE RESTRICT
37);
38
39 CREATE TABLE OrderDetails (
40 order_detail_id INTEGER PRIMARY KEY,
41 order_id INTEGER NOT NULL,
42 product_id INTEGER NOT NULL,
43 quantity INTEGER NOT NULL CHECK (quantity > 0),
44 unit_price_at_order NUMERIC NOT NULL CHECK (unit_price_at_order >= 0),
45 FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON UPDATE CASCADE ON DELETE CASCADE,
46 FOREIGN KEY (product_id) REFERENCES Products(product_id) ON UPDATE CASCADE ON DELETE RESTRICT,
47 UNIQUE (order_id, product_id)
48);
49
50 CREATE INDEX idx_orders_user_date ON Orders(user_id, order_date);
51 CREATE INDEX idx_orderdetails_order ON OrderDetails(order_id);
52 CREATE INDEX idx_orderdetails_product ON OrderDetails(product_id);
53 CREATE INDEX idx_orders_date ON Orders(order_date);
54 """
55
56 # Insert sample data
57 users_data = [('2026-03-18 16:30:00', 'delivered', 4, 1, 'Wireless Mouse', 1, 25, 25),
58 ('2026-03-05 10:00:00', 'paid', 1, 1, 'Wireless Mouse', 2, 25, 50),
59 ('2026-03-05 10:00:00', 'paid', 2, 3, 'USB-C Hub', 1, 45, 45)]
60
61 # Insert into Users table
62 cursor.execute('INSERT INTO Users (email, full_name, phone, created_at) VALUES (?, ?, ?, ?)')
63 for user in users_data:
64 cursor.execute('INSERT INTO Users (email, full_name, phone, created_at) VALUES (?, ?, ?, ?)')
65
66 # Insert into Orders table
67 cursor.execute('INSERT INTO Orders (order_id, user_id, order_date, status) VALUES (?, ?, ?, ?)')
68 for order in users_data:
69 cursor.execute('INSERT INTO Orders (order_id, user_id, order_date, status) VALUES (?, ?, ?, ?)')
70
71 # Insert into OrderDetails table
72 cursor.execute('INSERT INTO OrderDetails (order_detail_id, order_id, product_id, quantity, unit_price_at_order) VALUES (?, ?, ?, ?, ?)')
73 for order in users_data:
74 cursor.execute('INSERT INTO OrderDetails (order_detail_id, order_id, product_id, quantity, unit_price_at_order) VALUES (?, ?, ?, ?, ?)')
75
76 # Commit and close
77 cursor.commit()
78 cursor.close()
79
80 # Print the schema and data
81 print('Schema and Data:')
82 print('Users:')
83 for user in users_data:
84 print(user)
85
86 print('Orders:')
87 for order in users_data:
88 print(order)
89
90 print('OrderDetails:')
91 for order in users_data:
92 print(order)
93
94 # Close the connection
95 conn.close()

CHAT
HOSPITAL MANAGEMENT SYSTEM SCHEMA AND QUERIES
requirements:
• Tables: Users, Products, Orders, OrderDetails.
• Generate queries to:
• Retrieve all orders by a user.
• Find the most purchased product.
• Calculate total revenue in a given month.
• AI should also suggest normalization improvements and query optimization.
Expected Output:
• Complete schema with relationships + SQL queries for analytics.
give code in python without comments

import sqlite3
from datetime import datetime

schema_sql = """
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;

CREATE TABLE Users (
 user_id INTEGER PRIMARY KEY,
 email TEXT NOT NULL UNIQUE,
 full_name TEXT NOT NULL,
 phone TEXT UNIQUE,
 created_at TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE Products (
 product_id INTEGER PRIMARY KEY,
 sku TEXT NOT NULL UNIQUE,
 product_name TEXT NOT NULL,
 unit_price NUMERIC NOT NULL CHECK (unit_price >= 0),
 is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1)),
);
"""

orders_sql = """
CREATE TABLE Orders (
 order_id INTEGER PRIMARY KEY,
 user_id INTEGER NOT NULL,
 order_date TEXT NOT NULL,
 status TEXT NOT NULL CHECK (status IN ('pending', 'paid', 'shipped', 'delivered', 'cancelled', 'returned')),
 FOREIGN KEY (user_id) REFERENCES Users(user_id) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE OrderDetails (
 order_detail_id INTEGER PRIMARY KEY,
 order_id INTEGER NOT NULL,
 product_id INTEGER NOT NULL,
 quantity INTEGER NOT NULL CHECK (quantity > 0),
 unit_price_at_order NUMERIC NOT NULL CHECK (unit_price_at_order >= 0),
 FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON UPDATE CASCADE ON DELETE CASCADE,
 FOREIGN KEY (product_id) REFERENCES Products(product_id) ON UPDATE CASCADE ON DELETE RESTRICT,
 UNIQUE (order_id, product_id)
);

CREATE INDEX idx_orders_user_date ON Orders(user_id, order_date);
CREATE INDEX idx_orderdetails_order ON OrderDetails(order_id);
CREATE INDEX idx_orderdetails_product ON OrderDetails(product_id);
CREATE INDEX idx_orders_date ON Orders(order_date);
"""

Insert sample data
users_data = [('2026-03-18 16:30:00', 'delivered', 4, 1, 'Wireless Mouse', 1, 25, 25),
 ('2026-03-05 10:00:00', 'paid', 1, 1, 'Wireless Mouse', 2, 25, 50),
 ('2026-03-05 10:00:00', 'paid', 2, 3, 'USB-C Hub', 1, 45, 45)]

Insert into Users table
cursor.execute('INSERT INTO Users (email, full_name, phone, created_at) VALUES (?, ?, ?, ?)')
for user in users_data:
 cursor.execute('INSERT INTO Users (email, full_name, phone, created_at) VALUES (?, ?, ?, ?)')

Insert into Orders table
cursor.execute('INSERT INTO Orders (order_id, user_id, order_date, status) VALUES (?, ?, ?, ?)')
for order in users_data:
 cursor.execute('INSERT INTO Orders (order_id, user_id, order_date, status) VALUES (?, ?, ?, ?)')

Insert into OrderDetails table
cursor.execute('INSERT INTO OrderDetails (order_detail_id, order_id, product_id, quantity, unit_price_at_order) VALUES (?, ?, ?, ?, ?)')
for order in users_data:
 cursor.execute('INSERT INTO OrderDetails (order_detail_id, order_id, product_id, quantity, unit_price_at_order) VALUES (?, ?, ?, ?, ?)')

Commit and close
cursor.commit()
cursor.close()

Print the schema and data
print('Schema and Data:')
print('Users:')
for user in users_data:
 print(user)

print('Orders:')
for order in users_data:
 print(order)

print('OrderDetails:')
for order in users_data:
 print(order)

Close the connection
conn.close()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS
Python Deb...
PS C:\Users\shash> c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56' 722' '--' 'c:\Users\shash\AAC A 16.3.py' --
orders by user: 1
(4, '2026-04-02 09:25:00', 'paid', 6, 2, 'Mechanical Keyboard', 2, 80, 160)
(4, '2026-04-02 09:25:00', 'paid', 7, 3, 'USB-C Hub', 1, 45, 45)
(2, '2026-03-18 16:30:00', 'delivered', 3, 2, 'Mechanical Keyboard', 1, 80, 80)
(2, '2026-03-18 16:30:00', 'delivered', 4, 1, 'Wireless Mouse', 1, 25, 25)
(1, '2026-03-05 10:00:00', 'paid', 1, 1, 'Wireless Mouse', 2, 25, 50)

AACA 16.3.py X

C:\Users\shash> AAC A 16.3.py > ...
28
29 CREATE TABLE Orders (
30 order_id INTEGER PRIMARY KEY,
31 user_id INTEGER NOT NULL,
32 order_date TEXT NOT NULL,
33 status TEXT NOT NULL CHECK (status IN ('pending', 'paid', 'shipped', 'delivered', 'cancelled', 'returned')),
34 FOREIGN KEY (user_id) REFERENCES Users(user_id) ON UPDATE CASCADE ON DELETE RESTRICT
35);
36
37 CREATE TABLE OrderDetails (
38 order_detail_id INTEGER PRIMARY KEY,
39 order_id INTEGER NOT NULL,
40 product_id INTEGER NOT NULL,
41 quantity INTEGER NOT NULL CHECK (quantity > 0),
42 unit_price_at_order NUMERIC NOT NULL CHECK (unit_price_at_order >= 0),
43 FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON UPDATE CASCADE ON DELETE CASCADE,
44 FOREIGN KEY (product_id) REFERENCES Products(product_id) ON UPDATE CASCADE ON DELETE RESTRICT,
45 UNIQUE (order_id, product_id)
46);
47
48 CREATE INDEX idx_orders_user_date ON Orders(user_id, order_date);
49 CREATE INDEX idx_orderdetails_order ON OrderDetails(order_id);
50 CREATE INDEX idx_orderdetails_product ON OrderDetails(product_id);
51 CREATE INDEX idx_orders_date ON Orders(order_date);
52 """
53
54 # Insert sample data
55 users_data = [('2026-03-18 16:30:00', 'delivered', 4, 1, 'Wireless Mouse', 1, 25, 25),
56 ('2026-03-05 10:00:00', 'paid', 1, 1, 'Wireless Mouse', 2, 25, 50),
57 ('2026-03-05 10:00:00', 'paid', 2, 3, 'USB-C Hub', 1, 45, 45)]
58
59 # Insert into Users table
60 cursor.execute('INSERT INTO Users (email, full_name, phone, created_at) VALUES (?, ?, ?, ?)')
61 for user in users_data:
62 cursor.execute('INSERT INTO Users (email, full_name, phone, created_at) VALUES (?, ?, ?, ?)')
63
64 # Insert into Orders table
65 cursor.execute('INSERT INTO Orders (order_id, user_id, order_date, status) VALUES (?, ?, ?, ?)')
66 for order in users_data:
67 cursor.execute('INSERT INTO Orders (order_id, user_id, order_date, status) VALUES (?, ?, ?, ?)')
68
69 # Insert into OrderDetails table
70 cursor.execute('INSERT INTO OrderDetails (order_detail_id, order_id, product_id, quantity, unit_price_at_order) VALUES (?, ?, ?, ?, ?)')
71 for order in users_data:
72 cursor.execute('INSERT INTO OrderDetails (order_detail_id, order_id, product_id, quantity, unit_price_at_order) VALUES (?, ?, ?, ?, ?)')
73
74 # Commit and close
75 cursor.commit()
76 cursor.close()
77
78 # Print the schema and data
79 print('Schema and Data:')
80 print('Users:')
81 for user in users_data:
82 print(user)
83
84 print('Orders:')
85 for order in users_data:
86 print(order)
87
88 print('OrderDetails:')
89 for order in users_data:
90 print(order)
91
92 # Close the connection
93 conn.close()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS
Python Deb...
PS C:\Users\shash> c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56' 722' '--' 'c:\Users\shash\AAC A 16.3.py' --
(2, '2026-03-18 16:30:00', 'delivered', 4, 1, 'Wireless Mouse', 1, 25, 25)
(1, '2026-03-05 10:00:00', 'paid', 1, 1, 'Wireless Mouse', 2, 25, 50)
(1, '2026-03-05 10:00:00', 'paid', 2, 3, 'USB-C Hub', 1, 45, 45)

Most purchased product:
(1, 'Wireless Mouse', 6)

AAC A 16.3.py X

C: > Users > shash > AAC A 16.3.py > ...

```
55 INSERT INTO Users (email, full_name, phone) VALUES
56 ('alice@example.com', 'Alice Stone', '900000001'),
57 ('bob@example.com', 'Bob Ray', '900000002');
58
59 INSERT INTO Products (sku, product_name, unit_price) VALUES
60 ('SKU-100', 'Wireless Mouse', 25.00),
61 ('SKU-101', 'Mechanical Keyboard', 80.00),
62 ('SKU-102', 'USB-C Hub', 45.00);
63
64 INSERT INTO Orders (user_id, order_date, status) VALUES
65 (1, '2026-03-05 10:00:00', 'paid'),
66 (1, '2026-03-18 16:30:00', 'delivered'),
67 (2, '2026-03-20 12:10:00', 'shipped'),
68 (1, '2026-04-02 09:25:00', 'paid');
69
70 INSERT INTO OrderDetails (order_id, product_id, quantity, unit_price_at_order) VALUES
71 (1, 1, 2, 25.00),
72 (1, 3, 1, 45.00),
73 (2, 2, 1, 80.00),
74 (2, 1, 1, 25.00),
75 (3, 1, 3, 25.00),
76 (4, 2, 2, 80.00),
77 (4, 3, 1, 45.00);
78 """
79
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash> c;; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' '
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '
722' '--' 'c:\Users\shash\AAC A 16.3.py' ...
```

Total revenue for 2026-03:
275

Normalization improvements:
- Move order status values to an OrderStatus lookup table and reference by status_id.

AAC A 16.3.py X

C: > Users > shash > AAC A 16.3.py > ...

```
80 query_orders_by_user = """
81 SELECT
82     o.order_id,
83     o.order_date,
84     o.status,
85     od.order_detail_id,
86     p.product_id,
87     p.product_name,
88     od.quantity,
89     od.unit_price_at_order,
90     (od.quantity * od.unit_price_at_order) AS line_total
91 FROM Orders o
92 JOIN OrderDetails od ON od.order_id = o.order_id
93 JOIN Products p ON p.product_id = od.product_id
94 WHERE o.user_id = ?
95 ORDER BY o.order_date DESC, o.order_id, od.order_detail_id;
96 """
97
98 query_most_purchased_product = """
99 SELECT
100     p.product_id,
101     p.product_name,
102     SUM(od.quantity) AS total_units_sold
103 FROM OrderDetails od
104 JOIN Products p ON p.product_id = od.product_id
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash> c::; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '5722' '--' 'c:\Users\shash\AAC A 16.3.py' ...
```

Normalization improvements:

- Move order status values to an OrderStatus lookup table and reference by status_id.
- Split Products into ProductCatalog and ProductPricingHistory for time-based price changes.
- Add Address table and keep shipping/billing addresses per order in OrderAddress.
- Create ProductCategory and ProductCategoryMap for many-to-many category assignments.
- Add Payment and Shipment tables to avoid overloading Orders with transactional details.

AAC A 16.3.py X

C: > Users > shash > AAC A 16.3.py > ...

```
110 query_total_revenue_month = """
111 SELECT
112     COALESCE(SUM(od.quantity * od.unit_price_at_order), 0) AS total_revenue
113 FROM Orders o
114 JOIN OrderDetails od ON od.order_id = o.order_id
115 WHERE strftime('%Y-%m', o.order_date) = ?
116     AND o.status IN ('paid', 'shipped', 'delivered');
117 """
118
119 normalization_improvements = [
120     "Move order status values to an OrderStatus lookup table and reference by status_id.",
121     "Split Products into ProductCatalog and ProductPricingHistory for time-based price changes",
122     "Add Address table and keep shipping/billing addresses per order in OrderAddress.",
123     "Create ProductCategory and ProductCategoryMap for many-to-many category assignments.",
124     "Add Payment and Shipment tables to avoid overloading Orders with transactional details."
125 ]
126
127 query_optimization_suggestions = [
128     "Use composite index Orders(user_id, order_date) for user-order history queries.",
129     "Use covering index OrderDetails(product_id, quantity) to speed top-product aggregation.",
130     "Avoid function-wrapped date filters in production by storing month_key column (YYYY-MM) a",
131     "Use parameterized queries and prepared statements to improve plan reuse and safety.",
132     "For high scale analytics, maintain a daily sales summary table via ETL or incremental job",
133 ]
134
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS +

```
PS C:\Users\shash> c:: cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56
722' '--' 'c:\Users\shash\AAC A 16.3.py' ...
Query optimization suggestions:
- Use composite index Orders(user_id, order_date) for user-order history queries.
- Use covering index OrderDetails(product_id, quantity) to speed top-product aggregation.
- Avoid function-wrapped date filters in production by storing month_key column (YYYY-MM) and indexing i
t.
- Use parameterized queries and prepared statements to improve plan reuse and safety.
```

```
AAC A 16.3.py X
C: > Users > shash > AAC A 16.3.py > ...
135 def run_demo():
146
147     print("Orders by user:", user_id)
148     for row in orders:
149         print(row)
150
151     print("\nMost purchased product:")
152     print(top_product)
153
154     print("\nTotal revenue for", month_key + ":")
155     print(revenue)
156
157     print("\nNormalization improvements:")
158     for item in normalization_improvements:
159         print("-", item)
160
161     print("\nQuery optimization suggestions:")
162     for item in query_optimization_suggestions:
163         print("-", item)
164
165     conn.close()
166
167 if __name__ == "__main__":
168     run_demo()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS +

PS C:\Users\shash> c::; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56
722' '-.' 'c:\Users\shash\AAC A 16.3.py' ...
- Use covering index OrderDetails(product_id, quantity) to speed top-product aggregation.
- Avoid function-wrapped date filters in production by storing month_key column (YYYY-MM) and indexing i
t.
- Use parameterized queries and prepared statements to improve plan reuse and safety.
- For high scale analytics, maintain a daily sales summary table via ETL or incremental jobs.
PS C:\Users\shash>
```

Task 3 – Library Database

Task:

Design schema for a **Library Management System**.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:
 - Retrieve all books currently issued.
 - Find overdue books (loan date > 30 days).
 - Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

```
AAC A 16.3.py X
C:\Users\shash> AAC A 16.3.py ...
1
2 import sqlite3
3
4 schema_sql = """
5 PRAGMA foreign_keys = ON;
6
7 DROP TABLE IF EXISTS Loans;
8 DROP TABLE IF EXISTS Members;
9 DROP TABLE IF EXISTS Books;
10
11 CREATE TABLE Books (
12     book_id INTEGER PRIMARY KEY,
13     isbn TEXT NOT NULL UNIQUE,
14     title TEXT NOT NULL,
15     author TEXT NOT NULL,
16     category TEXT,
17     published_year INTEGER,
18     total_copies INTEGER NOT NULL CHECK (total_copies >= 0),
19     available_copies INTEGER NOT NULL CHECK (available_copies >= 0 AND available_copies <= total_copies)
20 );
21
22 CREATE TABLE Members (
23     member_id INTEGER PRIMARY KEY,
24     membership_no TEXT NOT NULL UNIQUE,
25     full_name TEXT NOT NULL,
26
27     email TEXT UNIQUE,
28     phone TEXT,
29     joined_date TEXT NOT NULL
30 );
31
32 CREATE TABLE Loans (
33     loan_id INTEGER PRIMARY KEY,
34     book_id INTEGER NOT NULL,
35     member_id INTEGER NOT NULL,
36     loan_date TEXT NOT NULL,
37     due_date TEXT NOT NULL,
38     return_date TEXT,
39     FOREIGN KEY (book_id) REFERENCES Books(book_id) ON UPDATE CASCADE ON DELETE RESTRICT,
40     FOREIGN KEY (member_id) REFERENCES Members(member_id) ON UPDATE CASCADE ON DELETE RESTRICT,
41     CHECK (date(due_date) >= date(loan_date)),
42     CHECK (return_date IS NULL OR date(return_date) >= date(loan_date))
43 );
44
45 CREATE INDEX idx_loans_book_id ON Loans(book_id);
46 CREATE INDEX idx_loans_member_id ON Loans(member_id);
47 CREATE INDEX idx_loans_return_due ON Loans(return_date, due_date);
48 CREATE INDEX idx_loans_loan_date ON Loans(loan_date);
49 CREATE INDEX idx_books_title_author ON Books(title, author);
50 CREATE INDEX idx_members_name ON Members(full_name);
51 """
52
53 def run():
54     conn = sqlite3.connect(":memory:")
55     conn.executescript(schema_sql)
56     conn.executescript(seed_sql)
57
58     print("Indexing Strategy:")
59     for s in indexing_strategy:
60         print("-", s)
61
62     print("\nAll books currently issued:")
63     for row in conn.execute(query_currently_issued):
64         print(row)
65
66     print("\nOverdue books (loan date > 30 days):")
67     for row in conn.execute(query_overdue_30_days):
68         print(row)
69
70     print("\nBooks loaned by each member:")
71     for row in conn.execute(query_books_loaned):
72         print(row)
73
74     conn.close()
75
76 if __name__ == "__main__":
77     run()
```

```
AAC A 16.3.py X
C:\Users\shash> AAC A 16.3.py ...
26     email TEXT UNIQUE,
27     phone TEXT,
28     joined_date TEXT NOT NULL
29 );
30
31 CREATE TABLE Loans (
32     loan_id INTEGER PRIMARY KEY,
33     book_id INTEGER NOT NULL,
34     member_id INTEGER NOT NULL,
35     loan_date TEXT NOT NULL,
36     due_date TEXT NOT NULL,
37     return_date TEXT,
38     FOREIGN KEY (book_id) REFERENCES Books(book_id) ON UPDATE CASCADE ON DELETE RESTRICT,
39     FOREIGN KEY (member_id) REFERENCES Members(member_id) ON UPDATE CASCADE ON DELETE RESTRICT,
40     CHECK (date(due_date) >= date(loan_date)),
41     CHECK (return_date IS NULL OR date(return_date) >= date(loan_date))
42 );
43
44 CREATE INDEX idx_loans_book_id ON Loans(book_id);
45 CREATE INDEX idx_loans_member_id ON Loans(member_id);
46 CREATE INDEX idx_loans_return_due ON Loans(return_date, due_date);
47 CREATE INDEX idx_loans_loan_date ON Loans(loan_date);
48 CREATE INDEX idx_books_title_author ON Books(title, author);
49 CREATE INDEX idx_members_name ON Members(full_name);
50 """
51
52 def run():
53     conn = sqlite3.connect(":memory:")
54     conn.executescript(schema_sql)
55     conn.executescript(seed_sql)
56
57     print("Indexing Strategy:")
58     for s in indexing_strategy:
59         print("-", s)
60
61     print("\nAll books currently issued:")
62     for row in conn.execute(query_currently_issued):
63         print(row)
64
65     print("\nOverdue books (loan date > 30 days):")
66     for row in conn.execute(query_overdue_30_days):
67         print(row)
68
69     print("\nBooks loaned by each member:")
70     for row in conn.execute(query_books_loaned):
71         print(row)
72
73     conn.close()
74
75 if __name__ == "__main__":
76     run()
```

AAC A 16.3.py X

C: > Users > shash > AAC A 16.3.py > ...

```
52 seed_sql = """
53 INSERT INTO Books (isbn, title, author, category, published_year, total_copies, available_copies
54 ('978-0140449136', 'The Odyssey', 'Homer', 'Epic', -700, 5, 2),
55 ('978-0451524935', '1984', 'George Orwell', 'Dystopian', 1949, 4, 1),
56 ('978-0743273565', 'The Great Gatsby', 'F. Scott Fitzgerald', 'Classic', 1925, 3, 0),
57 ('978-0061120084', 'To Kill a Mockingbird', 'Harper Lee', 'Classic', 1960, 6, 4);
58
59 INSERT INTO Members (membership_no, full_name, email, phone, joined_date) VALUES
60 ('M-1001', 'Asha Rao', 'asha@example.com', '9001111111', '2025-01-10'),
61 ('M-1002', 'Kiran Das', 'kiran@example.com', '9002222222', '2025-03-22'),
62 ('M-1003', 'Nina Paul', 'nina@example.com', '9003333333', '2025-06-05');
63
64 INSERT INTO Loans (book_id, member_id, loan_date, due_date, return_date) VALUES
65 (1, 1, date('now','-12 day'), date('now','+18 day'), NULL),
66 (2, 1, date('now','-40 day'), date('now','-10 day'), NULL),
67 (3, 2, date('now','-20 day'), date('now','+10 day'), NULL),
68 (4, 3, date('now','-50 day'), date('now','-20 day'), date('now','-5 day')),
69 (2, 3, date('now','-5 day'), date('now','+25 day'), NULL);
70 """
71
72 query_currently_issued = """
73 SELECT
74     l.loan_id,
75     b.book_id,
76     b.title,
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

+ v ... | C Python De

```
PS C:\Users\shash> c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '60
105' '--' 'c:\Users\shash\AAC A 16.3.py' --
```

```
Overdue books (loan date > 30 days):
(2, '1984', 'Asha Rao', '2026-02-20', '2026-03-22', 40)
```

```
Books loaned by each member:
(1, 'Asha Rao', 2)
(3, 'Nina Paul', 2)
```

1 Hidden

AAC A 16.3.py X

C: > Users > shash > AAC A 16.3.py > ...

```
74     l.loan_id,
75     b.book_id,
76     b.title,
77     b.author,
78     m.member_id,
79     m.full_name,
80     l.loan_date,
81     l.due_date
82 FROM Loans l
83 JOIN Books b ON b.book_id = l.book_id
84 JOIN Members m ON m.member_id = l.member_id
85 WHERE l.return_date IS NULL
86 ORDER BY l.loan_date DESC;
87 """
88
89 query_overdue_30_days = """
90 SELECT
91     l.loan_id,
92     b.title,
93     m.full_name,
94     l.loan_date,
95     l.due_date,
96     CAST(julianday('now') - julianday(l.loan_date) AS INTEGER) AS days_since_loan
97 FROM Loans l
98 JOIN Books b ON b.book_id = l.book_id
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

PS C:\Users\shash> c::; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\laur
105' '--' 'c:\Users\shash\AAC A 16.3.py' ...

Books loaned by each member:

```
(1, 'Asha Rao', 2)
(3, 'Nina Paul', 2)
(2, 'Kiran Das', 1)
```

PS C:\Users\shash>

AAC A 16.3.py X

C: > Users > shash > AAC A 16.3.py > ...

```
125 def run():
127     conn.executescript(schema_sql)
128     conn.executescript(seed_sql)
129
130     print("Indexing Strategy:")
131     for s in indexing_strategy:
132         print("-", s)
133
134     print("\nAll books currently issued:")
135     for row in conn.execute(query_currently_issued):
136         print(row)
137
138     print("\nOverdue books (loan date > 30 days):")
139     for row in conn.execute(query_overdue_30_days):
140         print(row)
141
142     print("\nBooks loaned by each member:")
143     for row in conn.execute(query_books_loaned_by_member):
144         print(row)
145
146     conn.close()
147
148 if __name__ == "__main__":
149     run()
150
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash> c::; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bur
105' '--' 'c:\Users\shash\AAC A 16.3.py' ...
```

Books loaned by each member:

```
(1, 'Asha Rao', 2)
(3, 'Nina Paul', 2)
(2, 'Kiran Das', 1)
```

PS C:\Users\shash> █

Task 4: Optimize Queries

Instructions:

- Use AI tools to analyze query efficiency.
- Optimize inefficient queries using joins, indexes, or reducing subqueries.
- Test optimized queries for correctness.

Starter Code Example:

-- Original query

```
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM
Authors WHERE country='UK');
```

-- Optimized query

```
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

Expected Output:

- Both queries return the same result: all books written by UK authors.
- Optimized query performs faster for larger datasets.

The screenshot shows a VS Code editor with a Python file named 'AAC A 16.3.py'. The script defines a function 'build_db' that creates a SQLite database with two tables: 'Authors' and 'Books'. The 'Authors' table has columns 'author_id' (INTEGER PRIMARY KEY), 'name' (TEXT NOT NULL), and 'country' (TEXT NOT NULL). The 'Books' table has columns 'book_id' (INTEGER PRIMARY KEY), 'title' (TEXT NOT NULL), 'author_id' (INTEGER NOT NULL), 'price' (REAL NOT NULL), and 'published_year' (INTEGER NOT NULL). A foreign key constraint is added between 'author_id' in 'Books' and 'author_id' in 'Authors'. The script also includes a chat window on the right side, which displays a query optimization example. The chat window shows a query: 'SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');'. It compares the original query with an optimized query: 'SELECT * FROM Books b JOIN Authors a ON b.author_id = a.author_id WHERE a.country = 'UK';'. The chat window also shows the expected output: 'Both queries return the same result: all books written by UK authors.' and 'Optimized query performs faster for larger datasets. give code in python without comments'.

```
def build_db(conn, author_count=20000, book_count=200000):
    cur = conn.cursor()
    cur.executescript("""
        PRAGMA journal_mode = MEMORY;
        PRAGMA synchronous = OFF;
        PRAGMA temp_store = MEMORY;
        DROP TABLE IF EXISTS Books;
        DROP TABLE IF EXISTS Authors;
        CREATE TABLE Authors (
            author_id INTEGER PRIMARY KEY,
            name TEXT NOT NULL,
            country TEXT NOT NULL
        );
        CREATE TABLE Books (
            book_id INTEGER PRIMARY KEY,
            title TEXT NOT NULL,
            author_id INTEGER NOT NULL,
            price REAL NOT NULL,
            published_year INTEGER NOT NULL,
            FOREIGN KEY (author_id) REFERENCES Authors(author_id)
        );
    """)
```

The screenshot shows a VS Code editor with a Python file named 'AAC A 16.3.py'. The script defines a function 'build_db' that creates a SQLite database with two tables: 'Authors' and 'Books'. The 'Authors' table has columns 'author_id' (INTEGER PRIMARY KEY), 'name' (TEXT NOT NULL), and 'country' (TEXT NOT NULL). The 'Books' table has columns 'book_id' (INTEGER PRIMARY KEY), 'title' (TEXT NOT NULL), 'author_id' (INTEGER NOT NULL), 'price' (REAL NOT NULL), and 'published_year' (INTEGER NOT NULL). A foreign key constraint is added between 'author_id' in 'Books' and 'author_id' in 'Authors'. The script also includes a chat window on the right side, which displays a query optimization example. The chat window shows a query: 'SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');'. It compares the original query with an optimized query: 'SELECT * FROM Books b JOIN Authors a ON b.author_id = a.author_id WHERE a.country = 'UK';'. The chat window also shows the expected output: 'Both queries return the same result: all books written by UK authors.' and 'Optimized query performs faster for larger datasets. give code in python without comments'.

```
def build_db(conn, author_count=20000, book_count=200000):
    countries = ["UK", "US", "IN", "CA", "AU", "DE", "FR", "JP", "BR", "ZA"]
    uk_bias = 0.15
    authors = []
    for i in range(1, author_count + 1):
        c = "UK" if random.random() < uk_bias else random.choice(countries[1:])
        name = "Author-" + str(i)
        authors.append((i, name, c))
    cur.executemany("INSERT INTO Authors(author_id, name, country) VALUES (?, ?, ?)", authors)
    books = []
    for i in range(1, book_count + 1):
        aid = random.randint(1, author_count)
        title = "Book " + "".join(random.choices(string.ascii_uppercase + string.digits, k=10))
        price = round(random.uniform(5, 200), 2)
        year = random.randint(1950, 2026)
        books.append((i, title, aid, price, year))
        if len(books) >= 5000:
            cur.executemany(
                "INSERT INTO Books(book_id, title, author_id, price, published_year) VALUES (?, ?, ?, ?, ?)",
                books
            )
            books.clear()
    if books:
        cur.executemany(
            "INSERT INTO Books(book_id, title, author_id, price, published_year) VALUES (?, ?, ?, ?, ?)",
            books
        )
```

AAC A 16.3.py X

C: > Users > shash > AAC A 16.3.py > ...

```
56 def create_indexes(conn):
57     conn.executescript("""
58     CREATE INDEX IF NOT EXISTS idx_authors_country_author_id ON Authors(country, author_id);
59     CREATE INDEX IF NOT EXISTS idx_books_author_id ON Books(author_id);
60     """)
61
62 def run_query(conn, sql, repeats=5):
63     cur = conn.cursor()
64     times = []
65     row_count = None
66     sample = None
67     for _ in range(repeats):
68         t0 = time.perf_counter()
69         rows = cur.execute(sql).fetchall()
70         t1 = time.perf_counter()
71         times.append(t1 - t0)
72         if row_count is None:
73             row_count = len(rows)
74             sample = rows[:5]
75     avg_ms = (sum(times) / len(times)) * 1000
76     return avg_ms, row_count, sample
77
78 def result_set_signature(conn, sql):
79     cur = conn.cursor()
80     s = set(cur.execute(sql).fetchall())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash> c::; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '61
013' '--' 'c:\Users\shash\AAC A 16.3.py'
```

Same results: True
Rows only in original: 0
Rows only in optimized: 0

Original sample rows: [(7614, 'Book_VTJPLZQ9RA', 5, 147.19, 1981), (18739, 'Book_ZGE9F33KSH', 5, 37.09,

AAC A 16.3.py X

C: > Users > shash > AAC A 16.3.py > ...

```
87 def main():
88     original_query = """
93     SELECT *
94     FROM Books
95     WHERE author_id IN (
96         SELECT author_id
97         FROM Authors
98         WHERE country = 'UK'
99     );
100     """
101
102     optimized_query = """
103     SELECT b.*
104     FROM Books b
105     JOIN Authors a ON b.author_id = a.author_id
106     WHERE a.country = 'UK';
107     """
108
109     t_orig_before, c_orig_before, _ = run_query(conn, original_query, repeats=3)
110     t_opt_before, c_opt_before, _ = run_query(conn, optimized_query, repeats=3)
111
112     create_indexes(conn)
113
114     t_orig_after, c_orig_after, sample_orig = run_query(conn, original_query, repeats=5)
115     t_opt_after, c_opt_after, sample_opt = run_query(conn, optimized_query, repeats=5)
116
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash> c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '61
013' '--' 'c:\Users\shash\AAC A 16.3.py'
2003), (54069, 'Book_61S15YZ6AO', 5, 137.3, 1972), (55424, 'Book_W6E6ADOE2F', 5, 126.16, 1966), (57441,
'Book_9CM28GE3EU', 5, 28.14, 2012)]
Optimized sample rows: [(7614, 'Book_VTJPLZQ9RA', 5, 147.19, 1981), (18739, 'Book_ZGE9F33KSH', 5, 37.09,
2003), (54069, 'Book_61S15YZ6AO', 5, 137.3, 1972), (55424, 'Book_W6E6ADOE2F', 5, 126.16, 1966), (57441,
'Book_9CM28GE3EU', 5, 28.14, 2012)]
```

Python Del

1 Hidden T

AAC A 16.3.py X

C: > Users > shash > AAC A 16.3.py > ...

```
87 def main():
120     same_result = set_orig == set_opt
121     only_orig = len(set_orig - set_opt)
122     only_opt = len(set_opt - set_orig)
123
124     plan_orig = explain_plan(conn, original_query)
125     plan_opt = explain_plan(conn, optimized_query)
126
127     print("Original count before indexes:", c_orig_before)
128     print("Optimized count before indexes:", c_opt_before)
129     print("Original avg ms before indexes:", round(t_orig_before, 3))
130     print("Optimized avg ms before indexes:", round(t_opt_before, 3))
131     print()
132     print("Original count after indexes:", c_orig_after)
133     print("Optimized count after indexes:", c_opt_after)
134     print("Original avg ms after indexes:", round(t_orig_after, 3))
135     print("Optimized avg ms after indexes:", round(t_opt_after, 3))
136     print()
137     print("Same results:", same_result)
138     print("Rows only in original:", only_orig)
139     print("Rows only in optimized:", only_opt)
140     print()
141     print("Original sample rows:", sample_orig)
142     print("Optimized sample rows:", sample_opt)
143     print()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash> c:: cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe'
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher
013' '--' 'c:\Users\shash\AAC A 16.3.py'
```

```
Original EXPLAIN QUERY PLAN:
(3, 0, 0, 'SEARCH Books USING INDEX idx_books_author_id (author_id=?)'
(7, 0, 0, 'LIST SUBQUERY 1')
(9, 7, 0, 'SEARCH Authors USING COVERING INDEX idx_authors_country_author_id (country=?)'
```

```
AAC A 16.3.py X
C: > Users > shash > AAC A 16.3.py > ...
87 def main():
137     print("Same results:", same_result)
138     print("Rows only in original:", only_orig)
139     print("Rows only in optimized:", only_opt)
140     print()
141     print("Original sample rows:", sample_orig)
142     print("Optimized sample rows:", sample_opt)
143     print()
144     print("Original EXPLAIN QUERY PLAN:")
145     for p in plan_orig:
146         print(p)
147     print()
148     print("Optimized EXPLAIN QUERY PLAN:")
149     for p in plan_opt:
150         print(p)
151
152     conn.close()
153
154 if __name__ == "__main__":
155     main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash> c::; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher013' '--' 'c:\Users\shash\AAC A 16.3.py'
(9, 7, 0, 'SEARCH Authors USING COVERING INDEX idx_authors_country_author_id (country=?)')

Optimized EXPLAIN QUERY PLAN:
(4, 0, 0, 'SEARCH a USING COVERING INDEX idx_authors_country_author_id (country=?)')
(8, 0, 0, 'SEARCH b USING INDEX idx_books_author_id (author_id=?)')
PS C:\Users\shash> 
```

Task 5: University Course Registration

Scenario:

SR University needs a **course registration system** for students enrolling in different courses under faculty supervision.

- Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.
- Define relationships:
 - Each student can register for multiple courses.
 - Each course is taught by a faculty member.
- Write SQL queries to:
 - List all students enrolled in a specific course.
 - Find faculty members teaching more than 2 courses.
 - Retrieve courses with the highest number of registrations.

```
AAC A 16.3.py X
C:\Users\shash> AAC A 16.3.py > ...
1 import sqlite3
2
3 schema_sql = """
4 PRAGMA foreign_keys = ON;
5
6 DROP TABLE IF EXISTS Registrations;
7 DROP TABLE IF EXISTS Students;
8 DROP TABLE IF EXISTS Courses;
9 DROP TABLE IF EXISTS Faculty;
10
11 CREATE TABLE Students (
12     student_id INTEGER PRIMARY KEY,
13     roll_no TEXT NOT NULL UNIQUE,
14     full_name TEXT NOT NULL,
15     email TEXT UNIQUE,
16     department TEXT NOT NULL,
17     joined_year INTEGER NOT NULL
18 );
19
20 CREATE TABLE Faculty (
21     faculty_id INTEGER PRIMARY KEY,
22     emp_no TEXT NOT NULL UNIQUE,
23     full_name TEXT NOT NULL,
24     email TEXT UNIQUE,
25     department TEXT NOT NULL
26 );
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash> c:\cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\
Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '57
630' '-.' 'c:\Users\shash\AAC A 16.3.py'
Students enrolled in CS101
('CS101', 'Database Systems', 1, 'SRU23CE001', 'Aarav Reddy', '2026-Spring')
('CS101', 'Database Systems', 2, 'SRU23CE002', 'Isha Kapoor', '2026-Spring')
('CS101', 'Database Systems', 3, 'SRU23CE001', 'Karan Das', '2026-Spring')
('CS101', 'Database Systems', 4, 'SRU22CE010', 'Meera Nair', '2026-Spring')
('CS101', 'Database Systems', 5, 'SRU22CE005', 'Nikhil Varma', '2026-Spring')
```

CHAT

HOSPITAL MANAGEMENT SYSTEM SCHEMA AND QUERIES

Scenario:
SR University needs a course registration system for students enrolling in different courses under faculty supervision.

- Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.
- Define relationships:
- Each student can register for multiple courses.
- Each course is taught by a faculty member.
- Write SQL queries to:
- List all students enrolled in a specific course.
- Find faculty members teaching more than 2 courses.
- Retrieve courses with the highest number of registrations.

give code in python without comments

```
import sqlite3

schema_sql = """
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS Registrations;
DROP TABLE IF EXISTS Students;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Faculty;

CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY
```

```
AAC A 16.3.py X
C:\Users\shash> AAC A 16.3.py > ...
28 CREATE TABLE Courses (
29     course_id INTEGER PRIMARY KEY,
30     course_code TEXT NOT NULL UNIQUE,
31     title TEXT NOT NULL,
32     credits INTEGER NOT NULL CHECK (credits > 0),
33     faculty_id INTEGER NOT NULL,
34     FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id) ON UPDATE CASCADE ON DELETE RESTRICT
35 );
36
37 CREATE TABLE Registrations (
38     registration_id INTEGER PRIMARY KEY,
39     student_id INTEGER NOT NULL,
40     course_id INTEGER NOT NULL,
41     semester TEXT NOT NULL,
42     registered_at TEXT NOT NULL DEFAULT (datetime('now')),
43     FOREIGN KEY (student_id) REFERENCES Students(student_id) ON UPDATE CASCADE ON DELETE RESTRICT
44     FOREIGN KEY (course_id) REFERENCES Courses(course_id) ON UPDATE CASCADE ON DELETE RESTRICT,
45     UNIQUE (student_id, course_id, semester)
46 );
47
48 CREATE INDEX idx_registrations_course_id ON Registrations(course_id);
49 CREATE INDEX idx_registrations_student_id ON Registrations(student_id);
50 CREATE INDEX idx_courses_faculty_id ON Courses(faculty_id);
51 """
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash> c:\cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\
Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '57
630' '-.' 'c:\Users\shash\AAC A 16.3.py'
Students enrolled in CS101
('CS101', 'Database Systems', 1, 'SRU23CE001', 'Aarav Reddy', '2026-Spring')
('CS101', 'Database Systems', 2, 'SRU23CE002', 'Isha Kapoor', '2026-Spring')
('CS101', 'Database Systems', 3, 'SRU23CE001', 'Karan Das', '2026-Spring')
('CS101', 'Database Systems', 4, 'SRU22CE010', 'Meera Nair', '2026-Spring')
('CS101', 'Database Systems', 5, 'SRU22CE005', 'Nikhil Varma', '2026-Spring')
```

CHAT

HOSPITAL MANAGEMENT SYSTEM SCHEMA AND QUERIES

```
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY,
    roll_no TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
    email TEXT UNIQUE,
    department TEXT NOT NULL,
    joined_year INTEGER NOT NULL
);

CREATE TABLE Faculty (
    faculty_id INTEGER PRIMARY KEY,
    emp_no TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
    email TEXT UNIQUE,
    department TEXT NOT NULL
);

CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY,
    course_code TEXT NOT NULL UNIQUE,
    title TEXT NOT NULL,
    credits INTEGER NOT NULL CHECK (credits > 0),
    faculty_id INTEGER NOT NULL,
    FOREIGN KEY (faculty_id) REFERENCES Faculty
);

CREATE TABLE Registrations (
```

AAC A 16.3.py X

C: > Users > shash > AAC A 16.3.py > ...

```
48 CREATE INDEX idx_registrations_course_id ON Registrations(course_id);
49 CREATE INDEX idx_registrations_student_id ON Registrations(student_id);
50 CREATE INDEX idx_courses_faculty_id ON Courses(faculty_id);
51 """
52
53 seed_sql = """
54 INSERT INTO Faculty (emp_no, full_name, email, department) VALUES
55 ('F1001', 'Dr. Priya Sharma', 'priya.sharma@sru.edu', 'CSE'),
56 ('F1002', 'Dr. Rahul Menon', 'rahul.menon@sru.edu', 'CSE'),
57 ('F1003', 'Dr. Ananya Rao', 'ananya.rao@sru.edu', 'ECE');
58
59 INSERT INTO Courses (course_code, title, credits, faculty_id) VALUES
60 ('CS101', 'Database Systems', 4, 1),
61 ('CS102', 'Data Structures', 4, 1),
62 ('CS201', 'Operating Systems', 4, 1),
63 ('CS202', 'Machine Learning', 3, 2),
64 ('EC101', 'Digital Electronics', 4, 3);
65
66 INSERT INTO Students (roll_no, full_name, email, department, joined_year) VALUES
67 ('SRU23CSE001', 'Aarav Reddy', 'aarav.reddy@sru.edu', 'CSE', 2023),
68 ('SRU23CSE002', 'Isha Kapoor', 'isha.kapoor@sru.edu', 'CSE', 2023),
69 ('SRU23ECE001', 'Karan Das', 'karan.das@sru.edu', 'ECE', 2023),
70 ('SRU22CSE010', 'Meera Nair', 'meera.nair@sru.edu', 'CSE', 2022),
71 ('SRU22ECE005', 'Nikhil Varma', 'nikhil.varma@sru.edu', 'ECE', 2022);
72
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash> c;; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe'
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher'
630' '--' 'c:\Users\shash\AAC A 16.3.py'
```

```
Faculty teaching more than 2 courses
(1, 'Dr. Priya Sharma', 3)
```

```
Courses with highest number of registrations
(1, 'CS101', 'Database Systems', 5)
```

AAC A 16.3.py X

C: > Users > shash > AAC A 16.3.py > ...

```
73 INSERT INTO Registrations (student_id, course_id, semester) VALUES
74 (1, 1, '2026-Spring'),
75 (1, 2, '2026-Spring'),
76 (1, 4, '2026-Spring'),
77 (2, 1, '2026-Spring'),
78 (2, 3, '2026-Spring'),
79 (3, 5, '2026-Spring'),
80 (3, 1, '2026-Spring'),
81 (4, 1, '2026-Spring'),
82 (4, 2, '2026-Spring'),
83 (4, 3, '2026-Spring'),
84 (5, 5, '2026-Spring'),
85 (5, 1, '2026-Spring');
86 """
87
88 query_students_in_course = """
89 SELECT
90     c.course_code,
91     c.title,
92     s.student_id,
93     s.roll_no,
94     s.full_name,
95     r.semester
96 FROM Registrations r
97 JOIN Students s ON s.student_id = r.student_id
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash> c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Sha
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\lib
630' '--' 'c:\Users\shash\AAC A 16.3.py'
```

```
Faculty teaching more than 2 courses
(1, 'Dr. Priya Sharma', 3)
```

```
Courses with highest number of registrations
(1, 'CS101', 'Database Systems', 5)
```

AAC A 16.3.py X

C: > Users > shash > AAC A 16.3.py > ...

```
139 def run():
140     conn = sqlite3.connect(":memory:")
141     conn.executescript(schema_sql)
142     conn.executescript(seed_sql)
143
144     course_code = "CS101"
145
146     print("Students enrolled in", course_code)
147     for row in conn.execute(query_students_in_course, (course_code,)):
148         print(row)
149
150     print("\nFaculty teaching more than 2 courses")
151     for row in conn.execute(query_faculty_more_than_2_courses):
152         print(row)
153
154     print("\nCourses with highest number of registrations")
155     for row in conn.execute(query_highest_registrations):
156         print(row)
157
158     conn.close()
159
160 if __name__ == "__main__":
161     run()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Users\shash> c::; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidh
\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\det
630' '--' 'c:\Users\shash\AAC A 16.3.py'
```

```
Faculty teaching more than 2 courses
(1, 'Dr. Priya Sharma', 3)
```

```
Courses with highest number of registrations
(1, 'CS101', 'Database Systems', 5)
```

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.